



Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

- Graph is a non linear data structure.
- Graph is defined as set of vertices and edges. Vertices (also called as Nodes) hold data, while edges connect vertices and represents relations between them.

$$G = \{ V , E \}$$

Where, $V = \{ A, B, C, D, E, F \}$

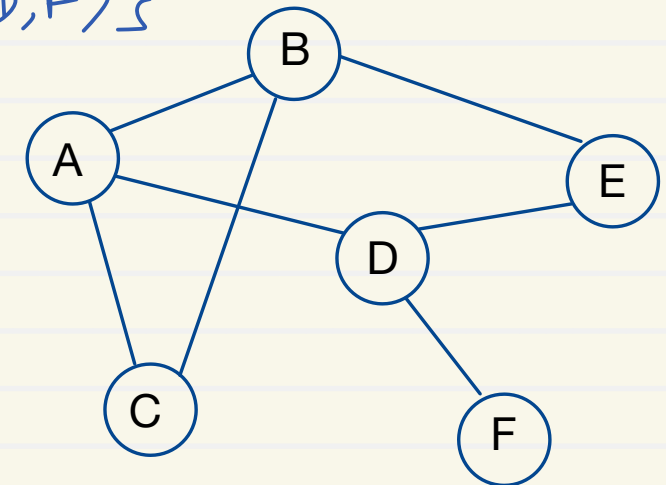
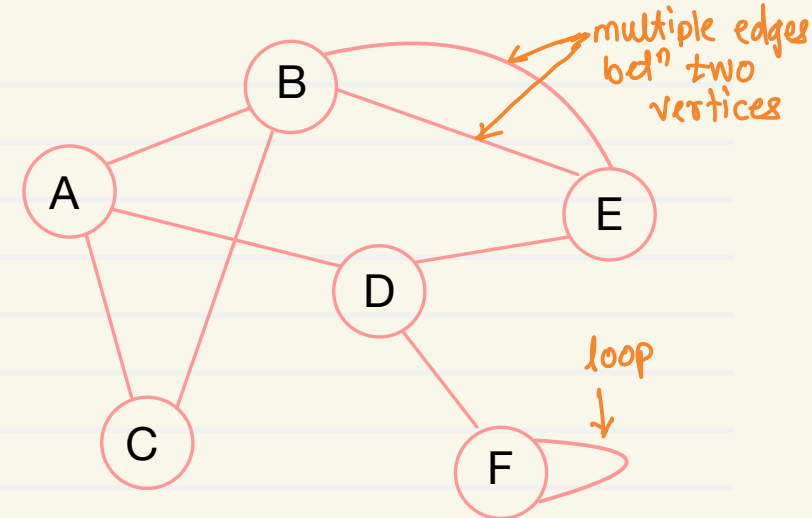
$E = \{ (A,B), (A,C), (A,D), (B,E), (B,C), (D,E), (D,F) \}$

- When there is an edge from vertex P to vertex Q, P is said to be adjacent to Q.



- Multi graph** - contains multiple edges in adjacent vertices or loops (edge connecting a vertex to it self)

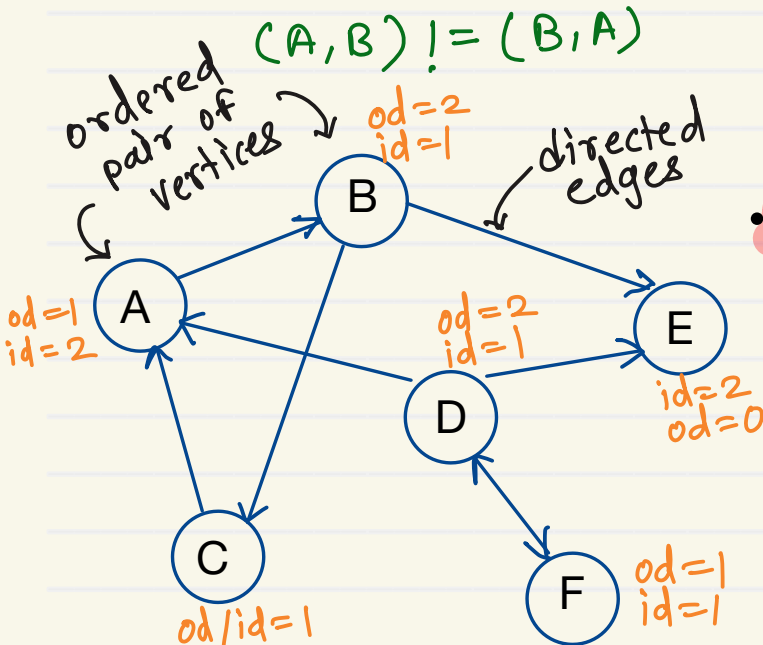
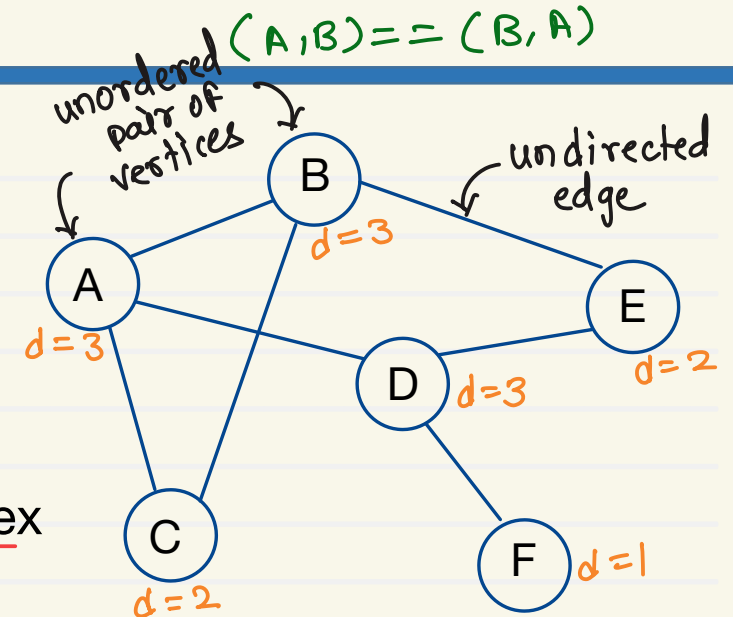
- Simple graph** - doesn't contains multiple edges in adjacent vertices or loops



- Graph edges may or may not have directions.

• Undirected graph : $G = \{ V, E \}$

- $V = \{ A, B, C, D, E, F \}$
- $E = \{ (A,B), (A,C), (A,D), (B,C), (B,E), (D,E), (D,F) \}$
- If A is adjacent to B, then B is also adjacent to A.
- Degree of vertex** : Number of vertices adjacent to the vertex



• Directed graph : $G = \{ V, E \}$

- $V = \{ A, B, C, D, E, F \}$
- $E = \{ \langle A,B \rangle, \langle C,A \rangle, \langle D,A \rangle, \langle B,C \rangle, \langle B,E \rangle, \langle D,E \rangle, \langle D,F \rangle, \langle F,D \rangle \}$
- If A is adjacent to B, then B may or may not be adjacent to A.
- Out degree** : Number of edges originated from the vertex
- In degree** : Number of edges terminated on the vertex

• **Path** : set of edges between two vertices. There can be

- A - D - E
- A - B - E
- A - C - B - E

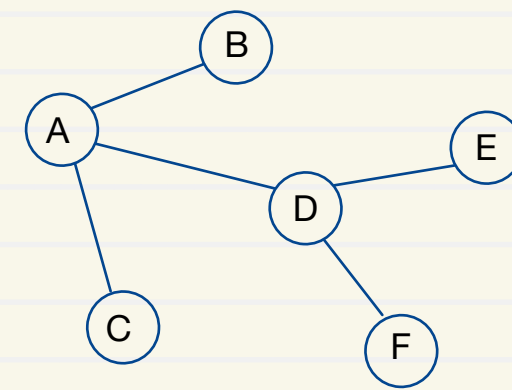
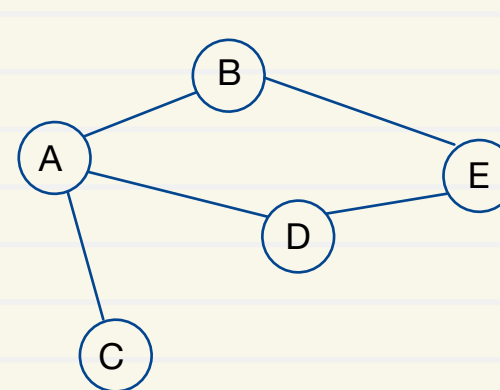
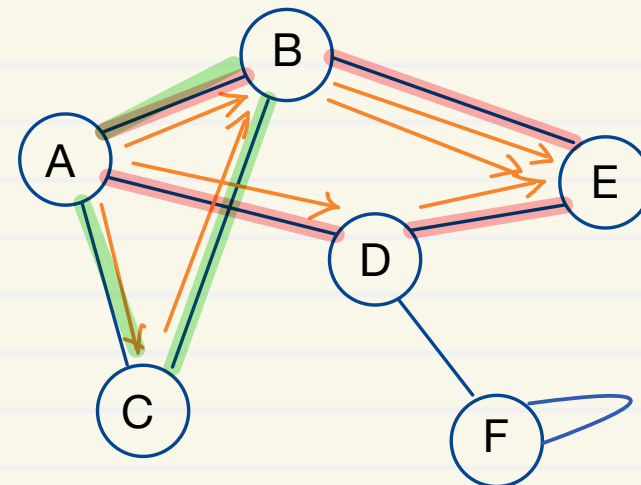
• **Cycle** : path whose start and end vertex is same.

- A - B - C - A
- A - B - E - D - A

• **Loop** : edge connecting vertex to itself. It is smallest cycle.

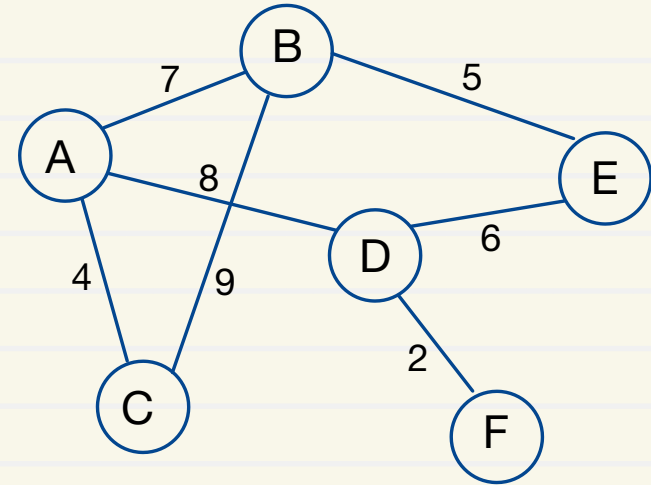
- F - F

• **Sub graph** : a graph having few vertices and few



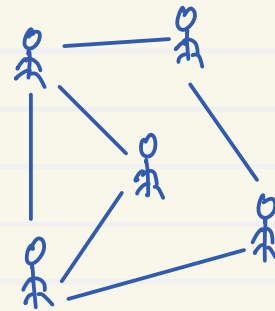
• **Weighted graph**

- graph edges have weight associated with them
- Weight represents some value eg distance,



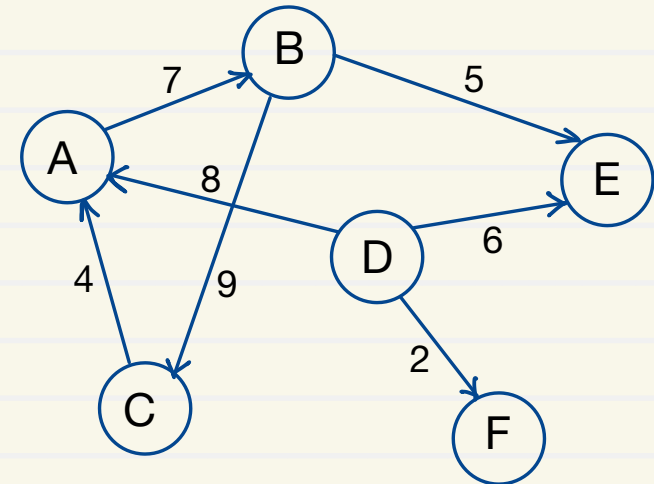
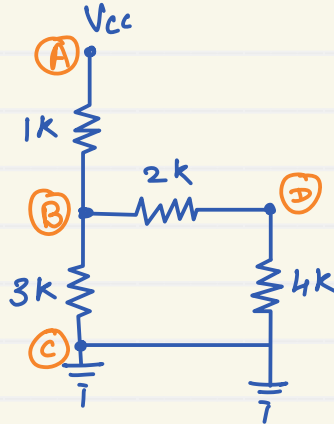
• **Directed weighted graph (Network)**

- Graph edges have directions as well as weights



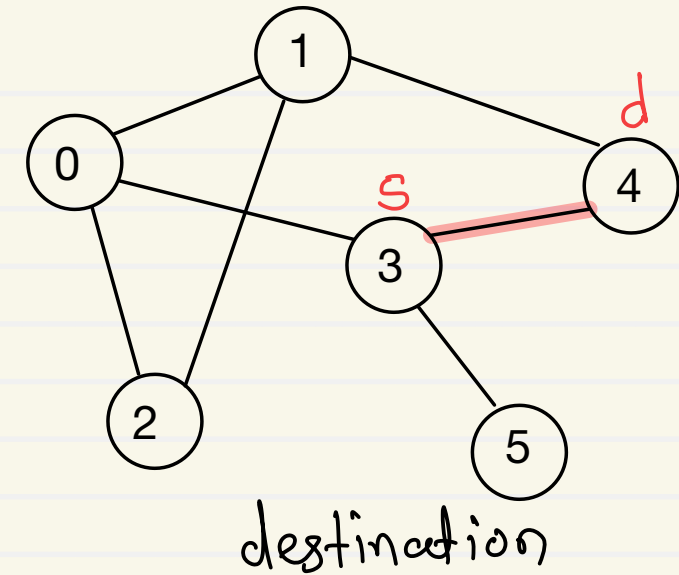
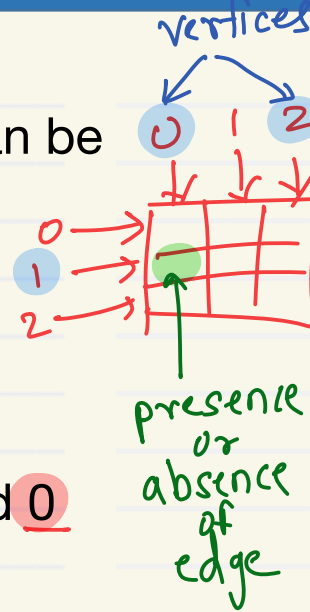
• **Graph applications**

- Electronic circuits
- Social media
- Communication network
- Road network
- Flight/Train/Bus services
- Bio-logical and chemical experiments
- Deep learning (Neural network, Tensor flow)
- Graph databases (Neo4j)



Graph implementation - Adjacency matrix

- If graph is having V vertices, a V x V matrix can be formed to store edges of the graph.
- Each matrix element represents presence or absence of the edge between vertices.
- For non weighted graph, 1 indicates edge and 0 indicates no edge.
- For un directed graph, adjacency matrix is always



destination

	0	1	2	3	4	5
0	0	1	1	1	0	0
1	1	0	1	0	1	0
2	1	1	0	0	0	0
3	1	0	0	0	1	1
4	0	1	0	1	0	0
5	0	0	0	1	0	0

source

- No. of vertices = 6
 - 6x6 Matrix is created to represent a graph

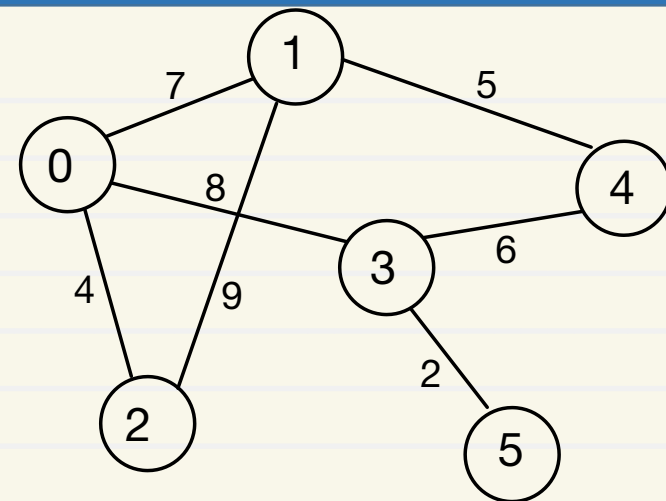
```
int adjMat[V][V];
boolean adjMat[V][V];
```

Graph implementation - Adjacency matrix

- If graph is having V vertices, a $V \times V$ matrix can be formed to store edges of the graph.
- Each matrix element represents presence or absence of the edge between vertices.
- For weighted graph, weight value indicates edge and infinity sign indicates no edge.
- For un directed graph, adjacency matrix is always

space complexity = $O(V^2)$
(implementation)

```
int adjMat[V][V];
```



destination

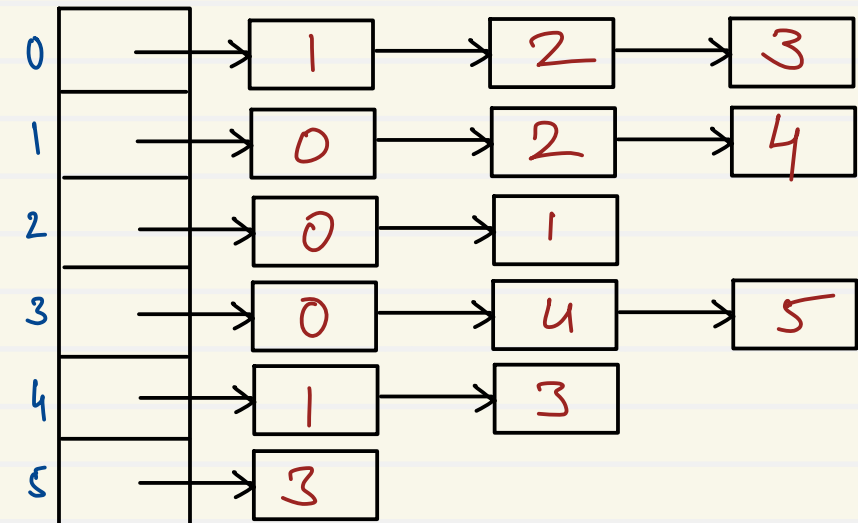
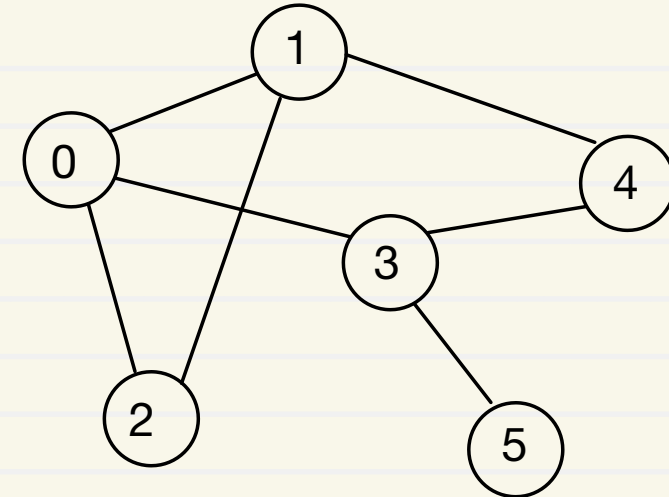
	0	1	2	3	4	5
0	∞	7	4	8	∞	∞
1	7	∞	9	∞	5	∞
2	4	9	∞	∞	∞	∞
3	8	∞	∞	∞	6	2
4	∞	5	∞	6	∞	∞
5	∞	∞	∞	2	∞	∞

source

Graph implementation - Adjacency list

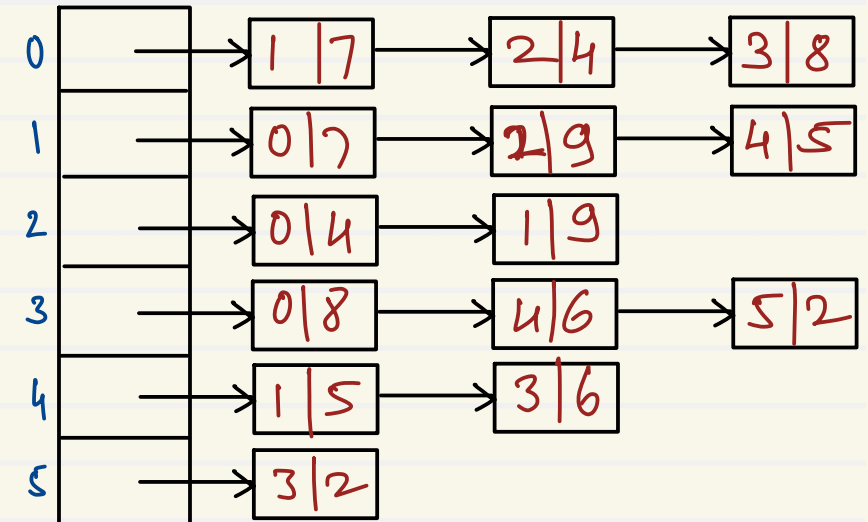
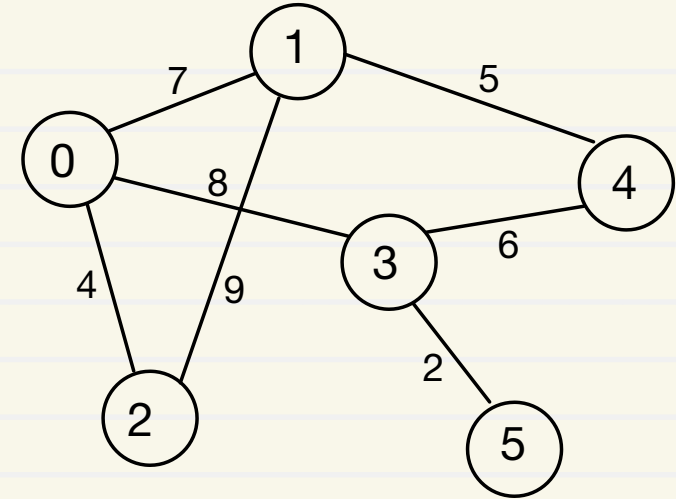
- Each vertex holds list of it's adjacent vertices.
- For non weighted graph, only neighbour vertices are stored.
- For weighted graph, neighbour vertices and weights of connecting edges are stored.
- If graph is sparse graph (with fewer number of edges), this implementation is more efficient (as

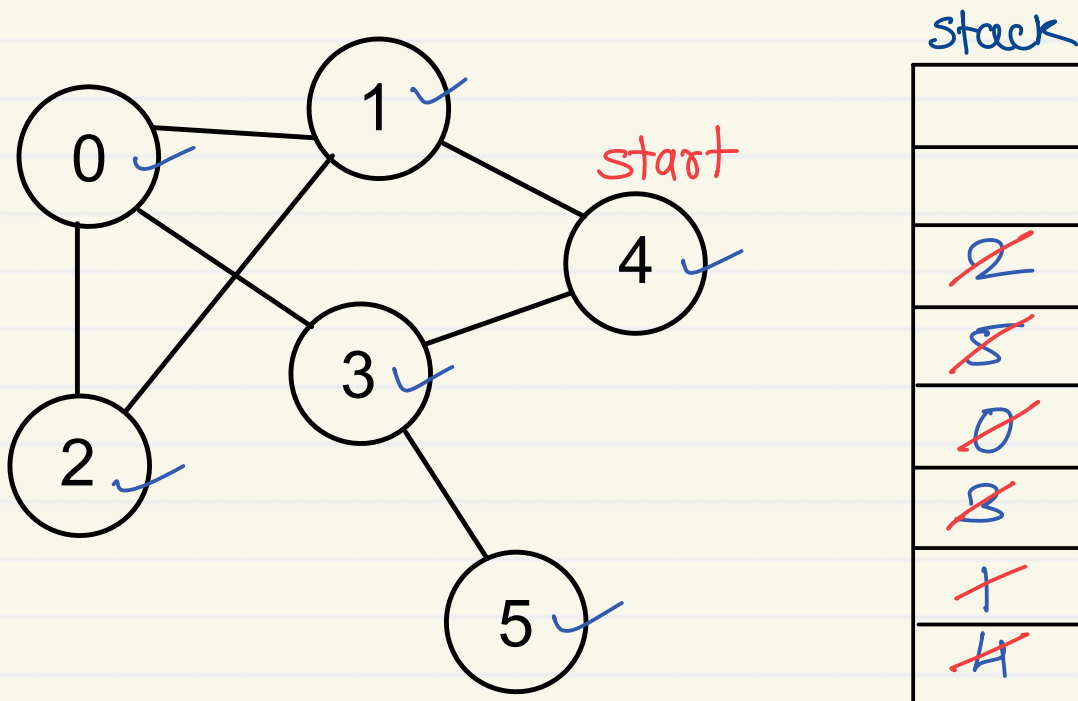
space complexity (implementation) = $O(V+E)$



Graph implementation - Adjacency list

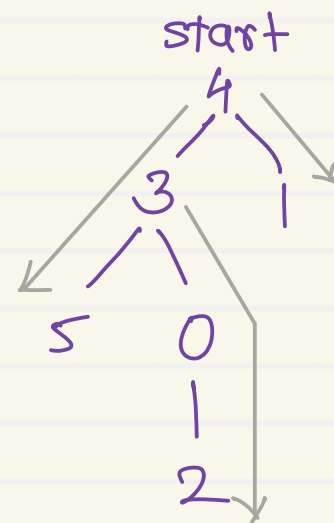
- Each vertex holds list of it's adjacent vertices.
- For non weighted graph, only neighbour vertices are stored.
- For weighted graph, neighbour vertices and weights of connecting edges are stored.
- If graph is sparse graph (with fewer number of edges), this implementation is more efficient (as

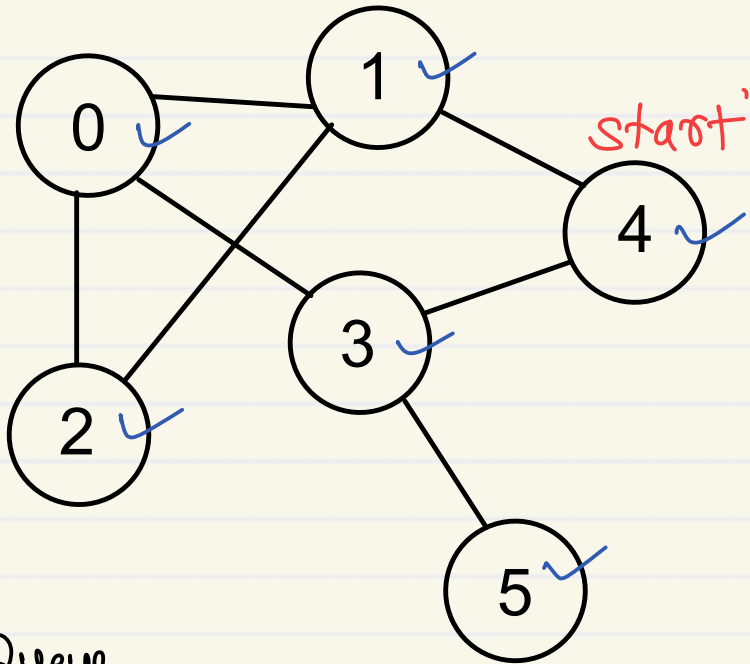




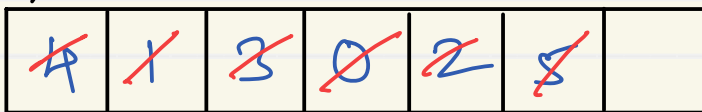
Traversal : 4, 3, 5, 0, 2, 1

1. Choose a vertex as start vertex.
2. Push start vertex on stack and mark it.
3. Pop vertex from stack.
4. Visit/print popped vertex.
5. Push all non marked neighbours of the vertex on the stack and mark them.
6. Repeat step 3 - 5 until stack is empty.





Queue



Traversal : 4, 1, 3, 0, 2, 5

1. Choose a vertex as start vertex.
2. Push start vertex on queue and mark it.
3. Pop vertex from queue .
4. Visit/print popped vertex.
5. Push all non marked neighbours of the vertex on the queue and mark them.
6. Repeat step 3 - 5 until queue is empty.

- BFS is also referred as level wise search algorithm

